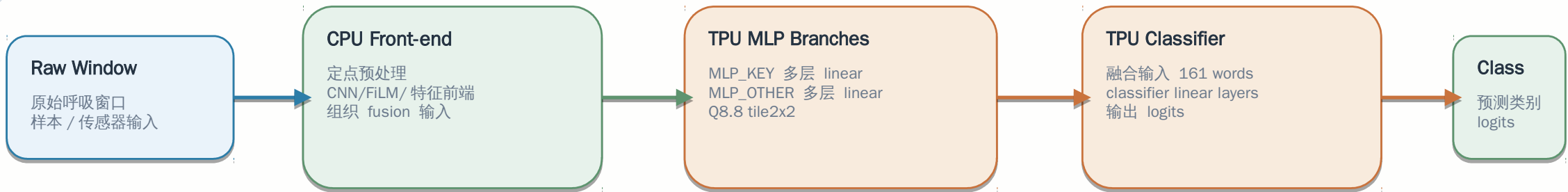


先从算法拆分讲起

呼吸识别模型不是一次性全部丢进 TPU，而是拆成 CPU 前端 + TPU MLP/Classifier 任务序列



为什么这样拆

TPU 当前已经验证的是 Q8.8 MLP/Classifier RTL。CNN/FILM 前端先放 CPU，避免把尚未验证的复杂卷积控制和数据复用一次性塞进硬件。

CPU 做什么

准备输入窗口和前端特征；把每个 TPU stage 的 input/param/output 地址写成 descriptor；通过 MMIO 发 launch。

TPU 做什么

按 descriptor 读 shared SRAM，把 input/param 流送入 MLP compute block，完成 linear/ReLU/round/saturate，再写回 output。

任务粒度

不是一次 launch 全模型。当前 demo 是多个 linear stage 依次 launch。

一句话

算法层面先把 CNN/FILM 前端留在 CPU，把最后 MLP/Classifier 拆成一串 Q8.8 linear 任务交给 TPU；CPU 负责调度和发 descriptor。

这样讲可以先回答“算法怎么拆”，再自然过渡到“CPU 怎么发给 TPU”。

CPU 怎么把一个 TPU 任务发出去

软件不是直接驱动 TPU 内部信号，而是写 shared SRAM 里的 descriptor/data，再写 TPU_CTRL MMIO 寄存器

descriptor + MMIO

1. 准备数据

CPU 把 input vector 写到 shared SRAM
参数池 param pool 已预加载或由 CPU 写入
输出区先清零

2. 写 descriptor

8 words:
net_id / input_addr / output_addr / param_addr
scratch_addr / input_words / output_words / flags

3. 同步内存

fence rw,rw
cache evict
settle loops
保证 TPU DMA 能看到最新数据

4. MMIO launch

写 MODE / NET_ID
写 DESC_LO = desc addr
写 CTRL.START

5. 等结果

轮询 STATUS
done/error
读 output words

descriptor 真实结构

```
TPUDesc {
  net_id, input_addr, output_addr, param_addr,
  scratch_addr, input_words, output_words, flags
}
flags[16] = TILE2X2_Q8_8, flags[0] = ReLU
```

MMIO 真实寄存器

```
TPU_CTRL_BASE = 0x4000_4000
DESC_LO @ 0x10
DESC_HI @ 0x14
CTRL.START bit0
STATUS busy/done/error
```

对应 C runtime 调用

```
tpu_select_desc_buffer()
tpu_build_desc()
tpu_submit_desc(): 写 desc_slot[0..7] + 写 TPU_REG_DESC_LO
+ 写 TPU_REG_CTRL START
tpu_wait_done(): 轮询 STATUS
```

重点口径

CPU 发给 TPU 的核心不是函数调用，而是“共享内存地址 + descriptor + MMIO START”。descriptor 告诉 TPU 去哪里读 input/param、去哪里写 output、这一层有多少 word。

TPU 收到 launch 后怎么执行

TPU_CTRL 只负责启动和状态；真正搬数据的是 descriptor DMA，真正算的是 Q8.8 MLP compute block



控制面路径

CPU DBUS -> splitter -> TPU_CTRL
地址窗口 0x4000_4000
只传控制寄存器和状态

数据面路径

TPU DMA 作为 AXI master
通过 shared_mem_subsys
访问 0x6000_0000 shared SRAM

计算路径

DMA 把 input/param 变成 valid
流
compute block 内部累加 Q16.16
输出 packed Q8.8 word

返回路径

DMA 写 output_addr
STATUS.done 置位
CPU poll 后读 output

讲解结论

所以这套接口本质上是一个轻量任务队列：CPU 写 descriptor，TPU DMA 按 descriptor 自己取数和写回，CPU 只负责启动、等待和读结果。